

UNIT III - Answer Key PDF

Collections and Object Oriented Programming

Teacher-only answer key. Do not share this PDF with students before results are published.

Section A - MCQ Answer Key

Question	Answer	Question	Answer	Question	Answer
Q1	A	Q2	B	Q3	C
Q4	C	Q5	B	Q6	C
Q7	A	Q8	C	Q9	B
Q10	B	Q11	A	Q12	B
Q13	C	Q14	B	Q15	C
Q16	C	Q17	B	Q18	D
Q19	B	Q20	B	Q21	C
Q22	C	Q23	A	Q24	A
Q25	B	Q26	B	Q27	C
Q28	B	Q29	B	Q30	A

Section B - Fill-up Answer Key

Q.No	Accepted Answer(s)
1	square
2	index
3	len()
4	append()
5	clear()
6	end
7	immutable
8	parentheses
9	changed
10	unique
11	intersection
12	union
13	key-value
14	keys()
15	values()
16	class
17	instance
18	methods
19	self
20	__init__

Section C - Simple Coding Answer Key

Q1. Write `list_sum(nums)`.

```
def list_sum(nums):  
    return sum(nums)
```

Evaluation notes: Loop-based solution also acceptable.

Q2. Write `max_mark(marks)`.

```
def max_mark(marks):  
    return max(marks)
```

Evaluation notes: Assume list is non-empty.

Q3. Write `reverse_list(items)` without changing original list.

```
def reverse_list(items):  
    return items[::-1]
```

Evaluation notes: Do not call `items.reverse()` and return the same list.

Q4. Write `tuple_first_last(t)`.

```
def tuple_first_last(t):  
    return (t[0], t[-1])
```

Evaluation notes: Assume tuple is non-empty.

Q5. Write `unique_count(items)`.

```
def unique_count(items):  
    return len(set(items))
```

Q6. Write `common_elements(a, b)` returning sorted common elements.

```
def common_elements(a, b):  
    return sorted(list(set(a) & set(b)))
```

Evaluation notes: Must return a list, not a set.

Q7. Write `get_student_mark(marks, name)`.

```
def get_student_mark(marks, name):  
    return marks.get(name, 'Not found')
```

Q8. Write `word_frequency(words)`.

```
def word_frequency(words):  
    freq = {}  
    for word in words:  
        freq[word] = freq.get(word, 0) + 1  
    return freq
```

Evaluation notes: Output dictionary must contain word counts.

Q9. Create class `Student` with `result()`.

```
class Student:  
    def __init__(self, name, mark):  
        self.name = name  
        self.mark = mark  
  
    def result(self):  
        if self.mark >= 35:  
            return 'Pass'  
        return 'Fail'
```

Evaluation notes: Must use `self` and store mark.

Q10. Create class `Rectangle` with `area()`.

```
class Rectangle:  
    def __init__(self, length, breadth):  
        self.length = length  
        self.breadth = breadth  
  
    def area(self):  
        return self.length * self.breadth
```

Evaluation notes: Must return product of length and breadth.